

C端海量用户自进化AI Agent平台技术设计说明书 (V1.0)

完全自研版 · 无任何外部云服务 · 无Kubernetes

文档信息

- 版本：1.0
 - 编制日期：2026年4月
 - 适用场景：面向亿级C端用户（App/Web/小程序/微信生态），峰值百万QPS、10万+同时在线Agent
 - 核心目标：每个用户拥有一个永久记忆、越用越强、永不Sleep的专属AI数字实体
 - 设计原则：100%忠实于自进化闭环思想，完全自研核心逻辑，基础设施全部自托管在自有服务器/数据中心（新加坡主数据中心 + 香港/东京灾备）
-

一、设计目标与约束

1.1 核心思想保留（完全脱离任何第三方Agent框架）

- 闭环自进化学习循环：每次用户交互后，系统主动提取可复用“技能”标准化YAML工作流），自动版本化、优化、持久化。下次类似任务直接调用，能力呈复利增长。复杂任务（≥5步工具调用或错误恢复）后自主复盘沉淀。
- 四层持久化内存系统：
 - 常驻提示层（核心知识 + 用户偏好）
 - 会话检索层（历史 + 全文 + 向量混合搜索）
 - 技能/过程记忆层（Git语义版本化工作流）
 - 用户建模层（动态知识图谱 + 重要性衰减）
- 自主技能沉淀：任务完成后自主生成标准化、可Fork/Merge的技能文件，支持全局脱敏共享 + A/B测试优化。
- Gateway与执行循环彻底解耦：接入层只负责多平台消息标准化，核心推理/工具循环完全异步、无状态。
- 工具执行隔离与并发控制：并行安全操作、路径锁定串行、破坏性操作沙箱化。

- 永久在线后台自主运行：支持事件驱动/Cron式离线任务，前端离线后Agent仍在后台进化。

1.2 硬性约束

- 完全自研：核心Agent Loop、内存引擎、技能提取器、学习工厂全部自主编码实现（Go + Python混合）。
 - 零外部云服务：无阿里云/AWS/GCP任何托管服务，无Serverless、API Gateway托管、Redis托管等。
 - 无Kubernetes：全部使用自托管开源组件 + 自研调度器。
 - 高可用目标：99.99% uptime（RTO<10s，RPO<1s）。
 - 高并发目标：峰值百万QPS，支撑亿级用户。
 - 部署环境：自有物理服务器/虚拟机集群（新加坡主DC + 异地多活灾备）。
-

二、整体系统架构

系统采用分层微服务 + 完全无状态执行 + 异步闭环设计，所有状态外置，执行层按需激活，避免任何常驻实例导致的资源爆炸。

text

C端用户（App/Web/微信/小程序）



[自研全球接入层] ← 自建Nginx/OpenResty集群 + 自研WebSocket Gateway (Go)



[协议标准化 + 认证限流层] ← 自研Go服务（JWT + Token Bucket + UserID一致性哈希路由）



[异步消息总线] ← 自托管Kafka集群（3 AZ，多分区，峰值百万QPS）



[自研任务调度器] ← 自研Go调度服务（基于etcd + Consul服务发现 + 自定义优先级队列）



[无状态Worker执行集群] ← 自研Go Worker进程（多节点水平扩展）

- 纯无状态：收到任务 → 动态加载用户记忆 → 执行一次完整推理+工具循环 → 写回状态 → 进程结束

- LLM推理：自托管vLLM/TGI集群 + 自研多模型路由器



[分布式内存与状态层] ← 自托管

- └─ 常驻提示层 → 自托管Redis Cluster（热缓存）

- └─ 会话检索层 → 自托管TiDB集群 + FTS5 + pgvector（Hybrid Search）

- └─ 技能层 → 自托管MinIO + DoltDB（Git语义版本仓库）

- └─ 用户建模层 → 自托管NebulaGraph集群 + 异步衰减Job

- └─ 全局技能市场 → 自托管Redis + Vector Index



[沙箱执行网格] ← 自研Firecracker MicroVM池 + 自研路径锁定服务（Redis Redlock）



[后台自进化工厂] ← 自研Flink-style流批一体Job集群（Ray自托管）



[可观测与自愈层] ← 自托管Prometheus + Grafana + Loki + 自研Chaos测试工具

三、各层详细设计

3.1 自研接入层

- 部署：多台物理服务器组成Active-Active集群，前置硬件负载均衡器（F5或自研L4 LB）。
- 功能：CDN加速 + WAF + WebSocket长连接 + 多平台协议适配（微信/Telegram/App统一Event对象）。
- 限流：自研Token Bucket（Per-User + 全局），超过阈值直接返回“思考中”。

3.2 异步消息总线

- 组件：自托管Kafka（3副本，ZooKeeper模式或KRaft）。
- Topic设计：user-input、agent-task、learning-job、skill-extract等。
- 削峰：所有非实时操作全部异步入队。

3.3 自研任务调度器

- 技术栈：Go + etcd（服务发现 + Leader选举） + Consul。
- 调度策略：优先级队列 + 公平调度 + 迭代预算控制（每个任务携带最大步骤数/Token上限）。
- 故障转移：Leader节点宕机后etcd自动选举新Leader。

3.4 无状态Worker执行集群

- 实现语言：Go（高性能） + Python（复杂推理部分）。
- 部署：多台高配服务器（CPU+GPU节点），每个节点运行固定数量Worker进程（systemd管理）。
- 工作流程（单次执行循环）：
 1. 从Kafka消费任务。
 2. 从Redis + TiDB毫秒级动态注入4层记忆。
 3. 执行自研Agent Loop（Reasoning → Tool Call → Observation）。
 4. 写回所有变更状态。
 5. 触发异步学习Job。
 6. 进程退出（零常驻）。
- 扩容方式：人工/脚本增加物理节点 + 自动注册到Consul。

3.5 刀布式内存与内存云

- 常驻提示层：Redis Cluster (MEMORY.yaml / USER.yaml热缓存, S3-like MinIO冷备)。
- 会话检索层：TiDB集群 (分布式SQL + FTS5 + pgvector混合检索)。
- 技能层：MinIO存储技能文件 + DoItDB提供Git-like版本控制 (Branch/Fork/Merge)。
- 用户建模层：NebulaGraph (知识图谱) + 每日异步衰减Job (重要性分数随时间/反馈指数衰减)。
- 全局技能市场：Redis + 自研向量索引, 支持脱敏后跨用户共享 + A/B测试。

3.6 沙箱执行网格

- 组件：自研Firecracker MicroVM管理器 (Rust实现, 轻量虚拟化)。
- 并发控制：Redis Redlock (同一路径写操作串行, 不同路径并行)。
- 所有工具调用 (代码执行、爬虫、文件操作) 必须进入临时MicroVM, 执行完毕自动销毁。

3.7 后台自进化工厂

- 组件：自托管Ray集群 (流批一体)。
 - 核心Job：
 - 技能提取器 (LLM总结 → YAML标准化)。
 - 记忆压缩Job (轻量规则替换 + 向量摘要)。
 - 用户画像更新 + 重要性衰减。
 - 全局技能A/B测试 + 优胜劣汰。
-

四、高可用与高并发策略

4.1 高可用

- 多AZ部署：新加坡主DC 3个机房 + 香港/东京灾备 (数据3副本Raft同步)。
- 状态恢复：每一步Reasoning/Action记录版本号 + Checkpoint, Worker失败后新进程从断点Resume (幂等设计)。
- LLM降级：自研路由器, 支持主模型 + 备选模型平滑切换。
- 自愈：自研心跳监控 + 自动重启脚本 + 每日Chaos Monkey测试。

4.2 高并发

- 完全无状态Worker：实例数只与实时并发任务数绑定，闲置用户零计算成本。
 - 状态动态注入：Redis热路径 + 向量预热，加载延迟<50ms。
 - 上下文优化：自研轻量压缩引擎（O(n)字符串替换 + 占位符），Token成本下降70%以上。
 - 迭代预算控制：内置Token/步骤上限，70%时注入“尽快收敛”提示，90%时强制结束。
 - 扩容方式：增加物理服务器节点，Consul自动发现。
-

五、安全、隐私与运营

- 端到端加密：AES-256 + 用户端密钥管理。
 - 隐私合规：用户一键导出全部记忆 + 技能（JSON + YAML包）。
 - 商业模式：Freemium（基础记忆 + 共享技能池），Premium（无限进化 + 优先Worker配额）。
 - 监控：自托管Prometheus + Grafana全链路指标 + 自定义Agent健康仪表盘。
-

六、部署与运维方案

- 服务器规格建议：
 - Worker节点：CPU 64核 + 512GB RAM + 8×A100 GPU（按需）。
 - 存储节点：高IOPS NVMe + MinIO。
 - 部署流程：Ansible自动化 + 自研GitOps工具。
 - 灰度发布：蓝绿切换 + 流量切分。
 - 成本结构：主要为服务器硬件 + 网络 + 电力，LLM推理自托管优化后远低于云服务。
-

七、潜在风险与应对

- 记忆膨胀：异步压缩 + 衰减 + 按需召回。
- 技能冲突：版本控制 + 用户反馈Loop + A/B测试。
- 冷启动：Redis预热 + 技能缓存。

- 运维复杂度：自研监控 + 自动化脚本 + 专职SRE团队。

本设计说明书已完整实现自进化闭环 + 四层记忆 + 自主技能 + 解耦执行的核心思想，同时彻底满足“完全自研、无外部云服务、无Kubernetes”的全部约束，可直接作为开发蓝图启动编码与硬件采购。

如需立即输出以下任意部分，请直接回复对应序号：

1. 核心Agent Loop伪代码（Go/Python）
2. 4层内存详细Schema与索引设计
3. 自研任务调度器完整架构与代码骨架
4. 技能提取与版本控制详细流程
5. 部署Ansible Playbook模板
6. 完整成本与硬件配置清单（新加坡机房）

本系统一旦落地，每个C端用户都将真正拥有一个活的、持续成长的AI数字实体。